HiRTOS: A Multi-core RTOS Written in SPARK Ada

J. Germán Rivera
Austin, TX, USA; email: jgrivera67@gmail.com

Abstract

This paper describes the design of HiRTOS *(High-Integrity RTOS), a simple real-time operating system kernel and separation kernel written in SPARK Ada. HiRTOS targets safety-critical and security-sensitive embedded software applications that run in multi-core microcontrollers.*

Keywords: RTOS, multi-core, high-integrity, Ada, SPARK.

1 Introduction

Although there are several popular RTOSes for embedded applications that run on small microcontrollers, most of them are not designed with high-integrity applications in mind, and as such are written in C, an unsafe language. So, it would be desirable to have an RTOS specifically designed for high-integrity applications, and written in a safer language, even if application code is written in C/C++.

HiRTOS (*High Integrity* RTOS) [1] is a small RTOS kernel and separation kernel for embedded single-core and multi-core platforms that have a memory protection unit (MPU). HiRTOS has a preemptive thread scheduler with fixed priorities and round-robin for threads with the same priority. On a multi-core platform, each core has its own thread scheduler and threads always execute in the same core where they were created. No HiRTOS resources are shared between CPUs and no communication/synchronization across CPU cores is supported by HiRTOS. Mutexes and condition variables [2] are the only synchronization primitives in HiRTOS, and they only support synchronizing threads running on the same core. HiRTOS mutexes support both priority inheritance and priority ceiling protocols [5]. Unlike traditional condition variables, HiRTOS condition variables can also be waited on while having interrupts disabled, not just while holding a mutex.

Given the fact that there is a separate instance of the HiRTOS scheduler per CPU, and there is no interaction between CPUs at the HiRTOS level, we just need to do formal verification of the HiRTOS thread scheduler for the single-core case.

HiRTOS is written in the SPARK subset of Ada [3], without using pointers (Ada access types). All RTOS objects such as threads, mutexes and condition variables are allocated internally by HiRTOS, from statically allocated internal object pools. These object pools are RTOS-private global arrays of the corresponding RTOS object types, sized at compile time via configuration parameters, whose values are application-specific. RTOS object handles provided to application code are just indices into these internal object arrays. No actual RTOS object pointers are exposed to application code. No dynamic allocation/deallocation of RTOS objects is supported and no static allocation of RTOS objects in memory owned by application code is supported either.

2 HiRTOS Overview

2.1 Major Design Decisions

- For API simplicity, inspired by the thread synchronization primitives of the C11 standard library [4], mutexes and condition variables are the only synchronization primitives in *HiRTOS*. Other synchronization primitives such as semaphores, event flags and message queues can be implemented on top of mutexes and condition variables.

- Unlike C11 mutexes, *HiRTOS* mutexes can change the priority of the thread owning the mutex. *HiRTOS* mutexes support both priority inheritance and priority ceiling [5].

- Unlike C11 condition variables, *HiRTOS* condition variables can also be waited on while having interrupts disabled, not just while holding a mutex.

- *HiRTOS* atomic levels can be used to disable the thread scheduler or to disable interrupts at and below a given priority or to disable all interrupts.

- In a multi-core platform, there is one *HiRTOS* instance per CPU Core. Each *HiRTOS* instance is independent of each other. No resources are shared between *HiRTOS* instances. No communication between CPU cores is supported by *HiRTOS*, so that the *HiRTOS* API can stay the same for both single-core and multi-core platforms. Inter-core communication would need to be provided outside of *HiRTOS*, using doorbell interrupts and mailboxes or shared memory, for example.

- Threads are bound to the CPU core in which they were created, for the lifetime of the thread. That is, no thread migration between CPU cores is supported.

Reprinted from Ada User Journal, Vol. 45(3), September 2024, with permission. Copyright is held by the author/owner(s).

- All RTOS objects such as threads, mutexes and condition variables are allocated internally by *HiRTOS* from statically allocated internal object pools. These object pools are just RTOS-private global arrays of the corresponding RTOS object types, sized at compile time via configuration parameters, whose values are application-specific. RTOS object handles provided to application code are just indices into these internal object arrays. No actual RTOS object pointers exposed to application code. No dynamic allocation/deallocation of RTOS objects is supported and no static allocation of RTOS objects in memory owned by application code is supported either.

- All application threads run in unprivileged mode. For each thread, the only writable memory, by default, is its own stack and global variables. Stacks of other threads are not accessible. MMIO space is only accessible to privileged code, by default. Application driver code, other than ISRs, must request access (read-only or read-write permission) to *HiRTOS* via a system call.

- Interrupt service routines (ISRs) are seen as hardware-scheduled threads that have higher priority than all software-scheduled threads. They can only be preempted by higher-priority ISRs. They cannot block waiting on mutexes or condition variables.

2.2 Separation Kernel Major Design Decisions

Besides being a fully functional RTOS, HiRTOS can be used as a separation kernel [6].

- In a multi-core platform, there is one separation kernel instance per CPU Core. Each instance is independent of each other. No resources are shared between separation kernel instances. No communication between CPU cores is supported, so that the separation kernel API can stay the same for both single-core and multi-core platforms. Inter-core communication would need to be provided outside of HiRTOS, using doorbell interrupts and mailboxes or shared memory, for example.

- Each separation-kernel instance consists of one or more partitions. A partition is a spatial and temporal separation/isolation unit on which a bare-metal or RTOS-based firmware binary runs. Each partition consists of one more disjoint address ranges covering portions of RAM and MMIO space that only that partition can access. Also, each partition has its own interrupt vector table, its own set of physical interrupts and its own global machine state. So, the firmware hosted in each partition has the illusion that it owns an entire physical machine, with is own set of of physical peripherals, dedicated memory and CPU core. The CPU core is time-sliced among the partitions running on the same separation kernel instance.

- Partitions are bound to the CPU core in which they were created. That is, no partition migration between CPU cores is supported.

- Partitions are created at boot time before starting the partition scheduler on the corresponding CPU core. Partitions cannot be destroyed or terminated.

- The separation kernel code itself runs in hypervisor privilege mode. All partitions run at a privilege lower than hypervisor mode. Partitions can communicate with the separation kernel via hypervisor calls and via traps to hypervisor mode triggered from special machine instructions such as WFI. The separation kernel can communicate with partitions, by forwarding interrupts targeted to the corresponding partition.

2.3 HiRTOS Code Architecture

To have wider adoption of an RTOS written in bare-metal Ada, providing a C/C++ programming interface is a must. Indeed, multiple interfaces or "skins" can be provided to mimic widely popular RTOSes such as FreeRTOS [7] and RTOS interfaces such as the CMSIS RTOS2 API [8]. As shown on figure 1, HiRTOS has a C/C++ interface layer that porvides a FreeRTOS skin and and a CMSIS RTOS2 skin. Both skins are implemented on top of a native C skin. The native C skin is just a thin C wrapper that consists of a C header file containing the C functions prototypes of the corresponding Ada subprograms of the SPARK Ada native interface of HiRTOS.

In addition to the C/C++ interface, HiRTOS should also provide an Ada runtime library (RTS) skin, as shown on figure 2, so that baremetal Ada applications that use Ada tasking features can run on top of HiRTOS. This can be especially useful, given the limited number of microcontroller platforms for which there is a bare-metal Ada runtime library available with the GNAT Ada compiler. an all platforms where is avaiable now or in the future.

HiRTOS has been architected to be easily portable to any multi-core microcontroller or bare metal platform for which a GNAT Ada cross compiler is available. All platform-dependent code is isolated in the HiRTOS porting layer, which provides platform-independent interfaces to the rest of the HiRTOS code. To avoid any depdendency on a platform-specific bare-metal Ada runtime library, provided by the compiler, HiRTOS sits on top of a platform-independent portable minimal Ada runtime library.

Figure 3 shows the major code components of HiRTOS. The HiRTOS code base is structured in three conceptual layers. The *HiRTOS API* layer, the *HiRTOS internals layer* and the *HiRTOS porting layer*.

The *HiRTOS API* layer contains the HiRTOS public interface components. The `HiRTOS_Interrupt_Handling` Ada package contains the services to be invoked from top-level interrupt handlers to notify HiRTOS of entering an exiting interrupt context. `HiRTOS_Memory_Protection` contains the services to protect ranges of memory and MMIO space. `HiRTOS_Thread` contains the services to create and manage threads.

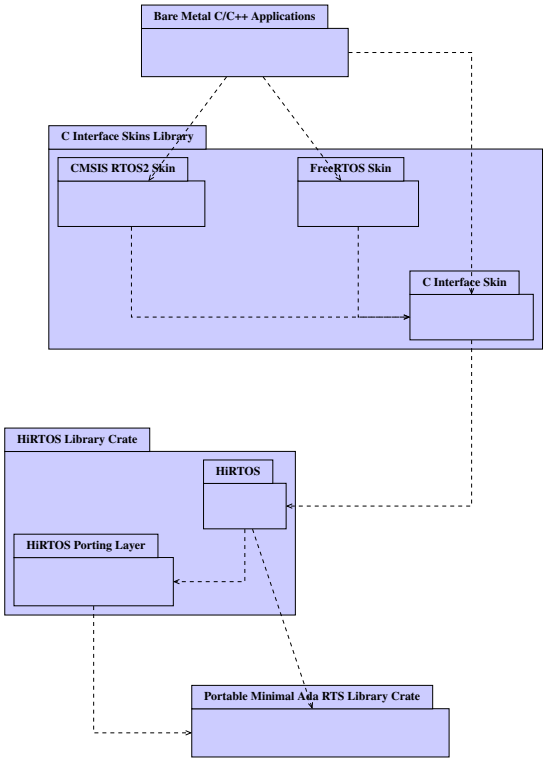

Figure 1: HiRTOS Code Architecture for C/C++ Applications

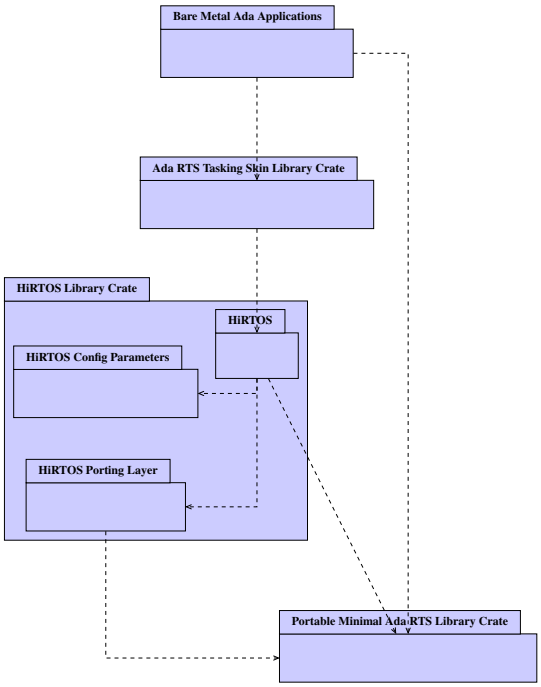

Figure 2: HiRTOS Code Architecture for Ada Applications

The *HiRTOS internals layer* contains HiRTOS-private components that are hardware-independent.

The *HiRTOS porting layer* contains hardware-dependent components that provide hardware-independent interfaces to upper HiRTOS layers.

References

[1] J. German Rivera, "HiRTOS: a high-integrity multi-core RTOS kernel and separation kernel written in SPARK Ada" `https://github.com/jgrivera67/HiRTOS`

[2] Andrew Birrell et al, "Synchronization primitives for a multiprocessor: a formal specification", Proceedings of the 11th Symposium on Operating System Principles, November, 1987

[3] John W. McCormick, Peter C. Chapin,"Building High Integrity Applications with SPARK", Cambridge University Press, 2015 `https://www.amazon.com/Building-High-Integrity-Applications-SPARK/dp/1107040736`

[4] ISO, "N2731: Working draft of the C23 standard, section 7.26", October 2021 `http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2596.pdf#page=345&zoom=100,102,113`

[5] Lui Sha et al, "Priority Inheritance Protocols: An Approach to Real-Time Synchronization", IEEE Transactions on Computers, September 1990 `https://www.csie.ntu.edu.tw/~r95093/papers/Priority%20Inheritance%20Protocols%20An%20Approach%20to%20Real-Time%20Synchronization.pdf`

[6] John Rushby, "Design and Verification of Secure Systems", ACM SIGOPS Operating Systems Review, 1981 `https://www.csl.sri.com/users/rushby/papers/sosp81.pdf`

[7] FreeRTOS `https://www.freertos.org/`

[8] CMSIS-RTOS API v2 (CMSIS-RTOS2) `https://www.keil.com/pack/doc/CMSIS/RTOS2/html/group__CMSIS__RTOS.html`

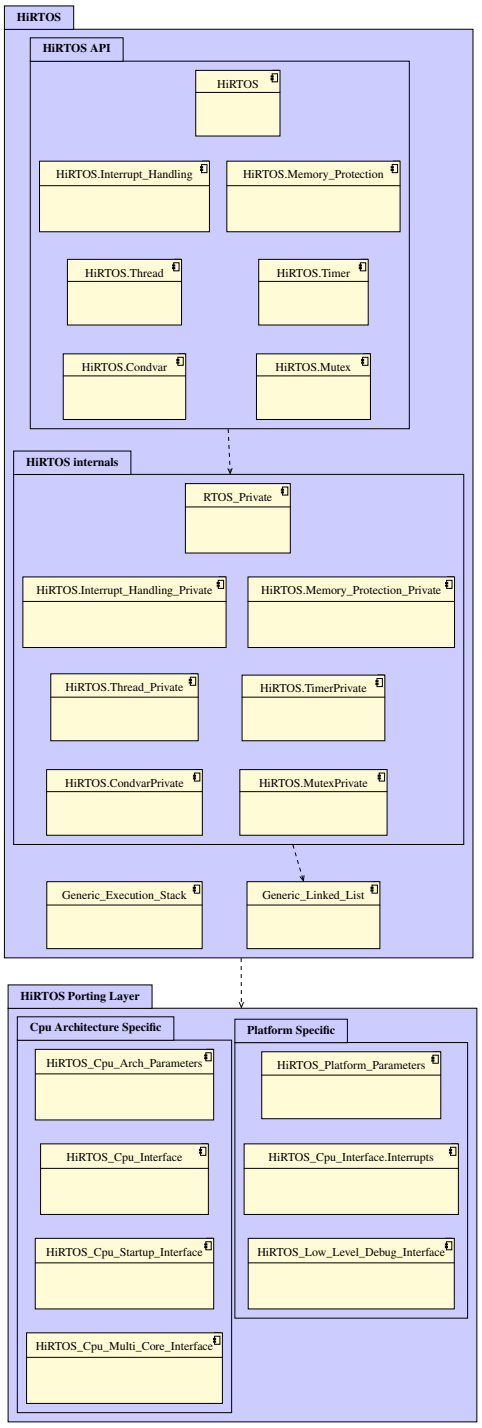

Figure 3: HiRTOS Code Components